

DOCUMENTACIÓN SISED

MODELO USUARIOS

1. Seguridad y Tipos de Usuarios (Control de Acceso)

El sistema será utilizado por **Recursos Humanos**, la **Comisión** y los **Aspirantes**.

Todos deben iniciar sesión, por lo que las credenciales se centralizan.

Tabla: ROLES

- ID_ROL (PK)
- NOMBRE_ROL (Ej: 'ASPIRANTE', 'RH', 'COMISION')

Tabla: USUARIOS

- ID_USUARIO (PK)
- ID_ROL (FK)
- CORREO_ELECTRONICO
- PASSWORD_HASH
- ESTATUS_ACTIVO (Boolean)

2. El Perfil del Docente (Datos Personales y de Origen)

La tabla ASPIRANTES es una extensión exclusiva para los usuarios con rol de docente.

Contiene la información general inicial solicitada.

Tabla: CATALOGO_CENTROS_TRABAJO

(Evita errores al capturar CCT o entidad federativa)

- ID_CCT (PK)
- CLAVE_CCT (Ej: '22DPT0001Z')
- NOMBRE_CCT (Ej: 'Instituto Tecnológico de Querétaro')
- ENTIDAD_FEDERATIVA (Ej: 'Querétaro')

Tabla: ASPIRANTES

- ID_ASPIRANTE (PK)

- ID_USUARIO (FK – Relación 1:1 con USUARIOS)
- NOMBRES
- APELLIDO_PATERNO
- APELLIDO_MATERNO
- RFC_HOMOCLAVE (Varchar, 13 caracteres)
- CURP (Varchar, 18 caracteres)
- ID_CCT_ACTUAL (FK a CATALOGO_CENTROS_TRABAJO)
- FECHA_INGRESO_SISTEMA (Date)

MODELO CATEGORIAS PROMOVIBLES

Esta estructura está diseñada para soportar el versionamiento inmutable de las normas y la evaluación lógica compleja de los requisitos.

1. Catálogos de Clasificación Base

Estas tablas definen las dimensiones sobre las cuales se construyen las categorías de promoción.

Tabla: CATALOGO_TIPOS_DOCENTE

- **ID_TIPO_DOCENTE** (PK): Identificador único.
- **NOMBRE**: Descripción del tipo (ej. Profesor Investigador, Técnico Docente).

Tabla: CATALOGO_NIVELES

- **ID_NIVEL** (PK): Identificador único.
- **CLAVE_NIVEL**: Identificador administrativo (ej. 'A', 'B', 'C').
- **SECUENCIA_NIVEL**: Orden jerárquico numérico para cálculos de "Nivel Inmediato Superior".

2. Entidad Central: Categorías Promovibles

Representa una plaza específica y su configuración legal. Utiliza el patrón de inmutabilidad por versiones.

Tabla: CATEGORIAS_PROMOVIBLES

- **ID_PROMOCION** (PK): Identificador de la versión específica de la categoría.
- **ID_TIPO_DOCENTE** (FK): Vínculo al catálogo de tipos.
- **ID_NIVEL** (FK): Vínculo al catálogo de niveles.
- **SUFIJO_CLAVE**: Parte variable de la clave presupuestal (ej. 'E3817').
- **TIENE_MEDIO_TIEMPO** (Boolean): Disponibilidad de plaza 20hrs.

- **TIENE_3/4_TIEMPO** (Boolean): Disponibilidad de plaza 30hrs.
- **TIENE_TIEMPO_COMPLETO** (Boolean): Disponibilidad de plaza 40hrs.
- **ARTICULO_REFERENCIA**: Texto del reglamento que fundamenta esta categoría.
- **VERSION** (Integer): Número correlativo de la versión de la regla.
- **FECHA_VIGENCIA_INICIO** (Timestamp): Entrada en vigor.
- **FECHA_VIGENCIA_FIN** (Timestamp): Fin de vigencia (Null si está activo).
- **ESTATUS_VERSION** (ENUM): 'VIGENTE', 'HISTORICA'.
- **ID_PROMOCION_PADRE** (FK): Referencia a la versión de la cual se originó el cambio.

3. Motor de Reglas Dinámico (Árbol Lógico)

Estas tablas permiten representar cualquier combinación de requisitos (Y/O) sin necesidad de programar lógica fija en el código.

Tabla: **GRUPOS_REQUISITOS**

Define la estructura del árbol lógico para una categoría.

- **ID_Grupo** (PK): Identificador del nodo en el árbol.
- **ID_PROMOCION** (FK): Relación con la categoría/versión específica.
- **ID_Grupo_Padre** (FK): Referencia recursiva para crear sub-nodos o ramas.
- **Tipo_Grupo** (ENUM): Determina la semántica del bloque (**RAIZ**, **INDISPENSABLE**, **ALTERNATIVA_UNO**, **ALTERNATIVA_DOS**, **SUBOPCION**).
- **Operador_Logico** (ENUM): Operación a aplicar a los hijos del grupo (**AND**, **OR**).

Tabla: **CONDICIONES_DETALLE**

Representa las "hojas" del árbol, donde reside la validación real contra los documentos.

- **ID_Condicion** (PK): Identificador de la regla individual.
- **ID_Grupo** (FK): Grupo al que pertenece la condición.
- **ID_Requisito** (FK): Apuntador al catálogo universal de parámetros (PO1, PO2, etc.).
- **Operador** (Varchar): Símbolo de comparación (ej. '=', '>=', 'CONTIENE').
- **Valor_Esperado** (Text): Instrucción detallada para la validación manual (ej. '2 años de antigüedad en el grado de doctor').

Tabla: **CATALOGO REQUISITOS CATEGORIAS**

- **ID_Requisito** (PK)
- **Código** (Text)
- **Nombre del Requisito** (Text)
- **Descripción** (Text)
- **Tipo Resumen Comprobante** (Text)

Relación Lógica del Modelo

El sistema funciona vinculando una **Convocatoria** a un **ID_PROMOCION** específico. Al momento de la validación, el motor recorre los **Grupos** desde la **RAIZ**, evaluando si las **Condiciones** asociadas han sido marcadas como "CUMPLE" en la tabla de evidencias, aplicando los **Operadores Lógicos** hacia arriba hasta determinar si el resultado final de la categoría es "Apto" o "No Apto".

¿Cómo funciona la lógica del SnapShot?

Cuando el administrador de Recursos Humanos entra al módulo de "Configuración de Requisitos" y decide cambiar el ">= 2 años" a ">= 3 años" en la categoría Titular B, tu código en Quarkus ejecutará el siguiente algoritmo transaccional:

1. **Congelamiento:** Hace un UPDATE al registro actual (Versión 1) en CATEGORIAS_PROMOVIBLES. Cambia su ESTATUS_VERSION a 'HISTORICA' y le pone la FECHA_VIGENCIA_FIN con la fecha y hora exacta de hoy.
2. **Clonación Maestra:** Hace un INSERT de un nuevo registro para Titular B. Le pone VERSION = 2, ESTATUS_VERSION = 'VIGENTE' y FECHA_VIGENCIA_INICIO = hoy. Esto genera un **nuevo ID_PROMOCION**.
3. **Clonación del Árbol de Reglas:** Tu backend lee todos los registros de la Versión 1 en las tablas GRUPOS_REQUISITOS y CONDICIONES, los duplica (clona) exactamente iguales, pero ahora apuntándolos al nuevo ID_PROMOCION de la Versión 2.
4. **La Mutación:** Finalmente, sobre esta nueva Versión 2 clonada, aplica el cambio de "3 años".

El Resultado Perfecto para las Convocatorias

Tu tabla transaccional CONVOCATORIAS_PLAZAS tiene el campo ID_PROMOCION (que es la llave foránea).

- La **Convocatoria de Enero** se quedó apuntando al ID_PROMOCION = 15 (La versión 1 con la regla de los 2 años). Aunque RH editó la regla en febrero, a esta convocatoria no le pasa absolutamente nada. El motor leerá la regla vieja sin problemas y dictaminará a Juan justamente.
- La **Convocatoria de Marzo** apuntará al ID_PROMOCION = 42 (La versión 2 con la regla de los 3 años). Los nuevos aspirantes serán evaluados con el rigor actualizado.

MODELO CONVOCATORIAS PARA PLAZAS

1. Los Nuevos Catálogos Satelitales

Para evitar que los usuarios escriban "Depto. de Sistemas", "Sistemas Computacionales" o "Ing. en Sistemas" para referirse a lo mismo, creamos estos diccionarios rígidos.

Tabla: CATALOGO_DEPARTAMENTOS

- ID_DEPARTAMENTO (PK)
- CLAVE (Varchar, ej: 'D-101')
- NOMBRE_DEPARTAMENTO (Varchar, ej: 'Sistemas y Computación')

Tabla: CATALOGO_AREAS_CONOCIMIENTO

- ID_AREA (PK)
- NOMBRE_AREA (Varchar, ej: 'Ingeniería y Tecnología')

Tabla: CATALOGO_DISCIPLINAS (Se relaciona con el Área para crear jerarquía)

- ID_DISCIPLINA (PK)
- ID_AREA (FK a CATALOGO_AREAS_CONOCIMIENTO)
- NOMBRE_DISCIPLINA (Varchar, ej: 'Ciencias de la Computación')

Tabla: CATALOGO_MATERIAS

- ID_MATERIA (PK)
- CLAVE_MATERIA (Varchar, si la tiene)
- NOMBRE_MATERIA (Varchar, ej: 'Estructura de Datos')

2. La Tabla Central de Plazas Ofertadas

Esta es la tabla transaccional donde Recursos Humanos dará de alta las vacantes que se abren en cada periodo.

Tabla: CONVOCATORIAS_PLAZAS

- ID_CONVOCATORIA (PK)
- NUM_CONVOCATORIA (Varchar, ej: 'CONV-2026-001' - *El identificador oficial*)
- DESCRIPCION_PLAZA (Text)
- CLAVE_PRESUPUESTAL_PLAZA (Varchar, ej: '1402 E3817 00 000000' - *La plaza exacta que libera el presupuesto*)

- ID_CATEGORIA (FK a CATEGORIAS_PROMOVIBLES - ¡El enlace de oro a tus reglas!)
- HORAS (Integer)
- ID_DEPARTAMENTO (FK a CATALOGO_DEPARTAMENTOS)
- PERFIL_REQUERIDO (Text)
- ID_DISCIPLINA (FK a CATALOGO_DISCIPLINAS - Al ligar la disciplina, implícitamente ya sabes el Área de Conocimiento)
- TEMA_A_EVALUAR (Text - Para la clase muestra o examen de oposición)
- SUELDO_M_07_MENSUAL (Decimal/Numeric - Para el tabulador exacto)
- ESTADO_CONVOCATORIA (ENUM: 'ABIERTA', 'EN_EVALUACION', 'CERRADA_DICTAMINADA', 'DESIERTA')

3. El Detalle de Materias (Relación N:M)

Como una plaza (especialmente si es de tiempo completo o 3/4 de tiempo) puede requerir que el docente imparta *más de una materia*, lo correcto a nivel de base de datos es hacer una tabla puente, en lugar de poner un solo campo de "Materia" en la tabla principal.

Tabla: CONVOCATORIA_MATERIAS_DETALLE

- ID_CONVOCATORIA (FK)
- ID_MATERIA (FK) (Llave primaria compuesta por ambos campos)

MOTIVOS DE RECHAZO

Creemos la tabla que almacenará las 32 filas de tu documento de texto. Le agregaremos una bandera booleana (REQUIERE_ESPECIFICACION) para controlar el comportamiento del motivo 32 en el frontend.

Tabla: CATALOGO_MOTIVOS_RECHAZO

- **ID_MOTIVO_RECHAZO** (PK)
- **CLAVE_MOTIVO** (Integer) -> *Los números del 1 al 32 de tu archivo.*
- **DESCRIPCION** (Varchar) -> *Ej: "NO CUBRE LOS REQUISITOS EN AÑOS DE TITULACIÓN."*
- **REQUIERE_ESPECIFICACION** (Boolean) -> *Será FALSE para los motivos 1 al 31, y TRUE únicamente para el motivo 32 ("OTROS").*

¿Cómo funciona esto en la vida real (Lógica de Negocio)?

1. **Rechazo Directo (Macro):** RH abre la solicitud de Juan Pérez. Se da cuenta de que Juan es Titular A y está pidiendo Titular C (se saltó la B). El revisor presiona el botón "Rechazar Solicitud". El sistema le despliega el catálogo. El revisor selecciona el **Motivo 29** y la solicitud completa pasa a estado RECHAZADA.
2. **Rechazo de un Documento (Micro):** RH está revisando una solicitud correcta, llega al archivo de la "Constancia de Nombramiento" y ve que no tiene el sello de Recursos Humanos. El revisor marca la evaluación como NO_CUMPLE. El sistema le exige un motivo. El revisor elige el **Motivo 26** ("PRESENTA DOCUMENTACIÓN SIN COTEJAR").
3. **La magia del Motivo 32:** Si el revisor marca NO_CUMPLE y selecciona el **Motivo 32** ("OTROS"), el backend (Quarkus) leerá que REQUIERE_ESPECIFICACION = TRUE en la base de datos, y el frontend (React) mostrará un cuadro de texto rojo obligatorio para que el validador escriba exactamente qué falló.

MODELO PROMOCIÓN

SOLICITUD

EVALUACIÓN

La Transacción: Solicitud de Evaluación (El Formulario)

Este es el equivalente digital a la hoja de papel que el aspirante entrega. Se vincula con tu tabla base `CATEGORIAS\_PROMOVIBLES` para saber a qué le está tirando.

Tabla: SOLICITUDES_PROMOCION

- ID_SOLICITUD (PK)
- ID_ASPIRANTE (FK)
- ID_CONVOCATORIA (FK a tu tabla `CONVOCATORIAS\_PLAZAS` - *La clave inmediata superior que solicita*)
- ALTERNATIVA_PARTICIPACION (ENUM: 'ALTERNATIVA_1', 'ALTERNATIVA_2')
- OPCION_ESCOLARIDAD (ENUM: 'LICENCIATURA', 'ESPECIALIDAD', 'MAESTRIA', 'DOCTORADO')
- SITUACION_ESCOLARIDAD (ENUM: 'TITULADO', 'SIN_GRADO', 'CON_GRADO')
- PROFESION_REGISTRADA (Varchar - *El texto libre que pide el recuadro*)
- FECHA_CREACION_SOLICITUD (Timestamp)
- ESTADO_SOLICITUD (ENUM: 'BORRADOR', 'ENVIADA', 'EN_REVISION', 'DICTAMINADA')
- ID_MOTIVO_RECHAZO_GENERAL (FK a CATALOGO_MOTIVOS_RECHAZO - Nullable).
- ESPECIFICACION_RECHAZO_GENERAL (Text - Nullable).

Tabla: CLAVES_ACTUALES_SOLICITUD (Plazas con las que concursa)

Soporta que el docente participe con una o varias plazas actuales.

- ID_CLAVE_ACTUAL (PK)
- ID_SOLICITUD (FK)
- CLAVE PRESUPUESTAL COMPLETA (Varchar - Campo único solicitado)
- TIPO_NOMBRAMIENTO (Integer: 10 o 95)
- EFECTOS_ANIO (Integer)
- EFECTOS_QUINCENA (Integer)

MODELO DE DOCUMENTOS EVIDENCIA

Llegamos a una de las partes más críticas e interesantes del sistema: el **Módulo de Evidencias**. Aquí es donde el mundo físico (los PDFs del aspirante) se conecta con el mundo lógico (nuestro motor de reglas y la solicitud).

1. Tabla: DOCUMENTOS_EVIDENCIA (El Archivo Físico)

Esta es la tabla central que almacena la metadata de cada archivo que sube el usuario.

- ID_DOCUMENTO (PK)
- ID_USUARIO (FK)
- ID_REQUISITO (FK a CATALOGO_REQUISITOS_CATEGORIA)
- NOMBRE_ARCHIVO
- URL_ALMACENAMIENTO

(Nota: Este registro solo representa que el PDF existe en el servidor).

2. Tabla: SOLICITUD_DOCUMENTOS_DETALLE (La Evaluación - "El Cómo")

Esta es la tabla donde Recursos Humanos hace su trabajo de validación manual regla por regla.

- ID_SOLICITUD (FK a la Solicitud A o B)
- ID_DOCUMENTO (FK al PDF del Título)
- ESTADO_EVALUACION (ENUM: 'PENDIENTE_REVISION', 'CUMPLE', 'NO_CUMPLE', 'OBSERVADO_PARA_SUBSANAR')
- FECHA_REVISION (Timestamp)
- ID_USUARIO_REVISOR (FK)
- ID_MOTIVO_RECHAZO (FK a CATALOGO_MOTIVOS_RECHAZO - Nullable. Solo se llena si ESTADO_EVALUACION es 'NO_CUMPLE').
- ESPECIFICACION_RECHAZO (Text - Nullable. Se llena obligatoriamente si el motivo elegido exige especificar, es decir, el motivo 32, o para dar contexto adicional al aspirante).

(Primary Key compuesta por ID_SOLICITUD + ID_DOCUMENTO)

El Algoritmo de Negocio (Cómo programarlo en Quarkus y React)

Para que la experiencia sea mágica para el usuario, el trabajo pesado lo hará tu backend en Java. Este es el flujo paso a paso cuando el aspirante hace clic en "Postularme":

1. **Selección Múltiple (Frontend):** El aspirante selecciona, por ejemplo, aplicar a la Convocatoria A y a la Convocatoria B al mismo tiempo.
2. **Deduplicación (Backend):** Quarkus consulta tu Motor de Reglas.
 - La Convocatoria A exige los requisitos: [P01, P02, P13].
 - La Convocatoria B exige los requisitos: [P01, P02, P28].

- Quarkus une las listas y elimina duplicados (usando un Set en Java): [P01 , P02 , P13 , P28].
- 3. **Carga Única (Frontend):** React le muestra al usuario exactamente 4 "cajas" para subir archivos. El usuario sube su Título (P01) una sola vez.
- 4. **Vinculación Inteligente (Backend):** Cuando el usuario le da a "Enviar", Quarkus guarda los 4 archivos físicos en DOCUMENTOS_EVIDENCIA. Inmediatamente después, inserta los registros en la tabla puente SOLICITUD_DOCUMENTOS_DETALLE:
 - Para la Solicitud A inserta: (Sol_A + Doc_P01), (Sol_A + Doc_P02), (Sol_A + Doc_P13).
 - Para la Solicitud B inserta: (Sol_B + Doc_P01), (Sol_B + Doc_P02), (Sol_B + Doc_P28).

El Impacto Operativo para Recursos Humanos

1. El validador de RH abre la **Solicitud A**. Ve el PDF del título y la regla: " ≥ 2 años". El título tiene 3 años. El validador marca **CUMPLE** en el registro de la tabla puente para la Solicitud A.
2. El validador (o otro compañero) abre la **Solicitud B**. Ve el *mismo* PDF del título y la regla: " ≥ 10 años". El título tiene 3 años. El validador marca **NO CUMPLE** en el registro puente de la Solicitud B y le pone el comentario: "*Tu grado solo tiene 3 años de antigüedad, no los 10 requeridos para esta plaza*".

MÓDULO DE AUDITORÍA GLOBAL

Implementar una bitácora transaccional (Audit Log) en un sistema federal no es solo una buena práctica, es un **requisito legal** para proteger a la institución ante auditorías de la Función Pública o inconformidades de los aspirantes.

Aquí tienes la implementación exacta:

1. El Modelo de Datos: BITACORA_TRANSACCIONAL

Esta tabla funcionará como una "caja negra" de avión. Registrará absolutamente todo lo que mute el estado del sistema.

Tabla: BITACORA_TRANSACCIONAL

- **ID_BITACORA** (PK, BigInt o UUID) -> Suele crecer muchísimo, por lo que un tipo de dato grande es ideal.
- **NOMBRE_TABLA** (Varchar) -> Ej: 'SOLICITUD_DOCUMENTOS_DETALLE' o 'SOLICITUDES_PROMOCION'.
- **ID_REGISTRO_AFECTADO** (Varchar) -> El ID de la fila que sufrió el cambio. Lo dejamos como Varchar por si auditas tablas que tienen llaves compuestas o UUIDs.
- **TIPO_ACCION** (ENUM: 'INSERT', 'UPDATE', 'DELETE')
- **VALORES_ANTERIORES** (JSONB) -> Cómo estaba la fila ANTES del cambio (NULL si es un INSERT).
- **VALORES_NUEVOS** (JSONB) -> Cómo quedó la fila DESPUÉS del cambio (NULL si es un DELETE).
- **ID_USUARIO** (FK a USUARIOS) -> El responsable de la acción. Si lo hizo el sistema automáticamente, puede ser un ID de "Usuario Sistema".
- **DIRECCION_IP** (Varchar) -> Desde qué computadora/red se hizo el cambio.
- **FECHA_ACCION** (Timestamp) -> Hora exacta de la modificación.

Implementación Técnica en tu Stack (Quarkus + PostgreSQL)

A nivel de Backend (Hibernate Envers / JPA EntityListeners)

Como usas Quarkus y Java, puedes aprovechar el ecosistema de Hibernate.

Puedes usar anotaciones como `@PreUpdate`, `@PrePersist` y `@PreRemove` en tus Entidades. Cuando Quarkus detecta que vas a guardar algo, intercepta la petición, extrae el `ID_USUARIO` del token JWT actual, serializa el objeto viejo y nuevo a JSON, e inserta en la bitácora.

- *Ventaja:* Tienes acceso total a la IP, al usuario logueado, y mantienes la lógica centralizada en Java.
- *Desventaja:* Si alguien hace un update directo por consola SQL, esquivará esta auditoría.